MATHEMATICAL PROOF-LANGUAGE DESIGN / ROUND FOUR

Rowan

Contract-Directed Mathematical Proofs

Property-aware elaboration, scoped reasoning, and admissible behavioral types over Lean

---

Central proposal. An object's type should expose the operations that its *current evidence* licenses. A proof step should either produce the required evidence for the intended mathematical objects, or expose a precise frontier of unfinished obligations. An abstract behavioral counterexample should count only when it describes a possible input.

Rowan preserves the round-three semantic core and makes three specific commitments beyond it: isolated proof requests with fixed meanings, an explicitly bounded rule-generation algorithm, and a complete affine-equality criterion on supplied semilinear observation images. A small parsed reference model and its executed adversarial tests accompany the article.

Prepared for Vladimir Reshetnikov

OpenAI | 6 September 2026

Evidence boundary. This is a language design, a set of written mathematical arguments, and an executed Python reference model. No Rowan Lean elaborator, new kernel extension, or human productivity study is claimed. A Lean toolchain was not available in this session. Earlier reports' kernel checks are attributed to those reports, not presented as new executions.

Abstract

Mathematical concision is not the omission of difficult premises. It is the ability to use an object through the properties already established about it, to recognize the contract of the next operation, and to reconstruct routine connections without restating them. Rowan proposes a property- and behavior-aware authoring layer over Lean organized around that principle. Its local types enrich fixed terms with evidence; its function interfaces state relational contracts; and its computation interfaces distinguish exact results, restricted equalities, observations, and newly constructed witnesses. All successful mathematical work is intended to elaborate to ordinary Lean.

The two round-three syntheses agree that these foundations are substantially settled. Rowan therefore concentrates on three unresolved boundaries rather than proposing another large grammar. First, a sealed proof request separates the meaning of a statement from the private metavariables used to prove it. Second, a finite, typed term pool makes theorem instantiation itself bounded, not merely the subsequent closure computation. Third, an admissibility-indexed behavioral type records the actual image of an observation. For affine length models over a supplied finite union of linear sets, a base-and-generator criterion decides universal equality and constructs a failing observation when equality fails. This handles empty element types and correlated lengths without confusing an impossible abstract input with a counterexample.

Worked developments preserve the locality of ProveIt's autonomous derivative lemma, the exact-division obligations of the Frey curve, and the finite/free premises and naturality of a tensor-family construction. The article specifies a certified CAS boundary, a proof-linked Leant handoff, a conservative translation, and a matched Lean baseline. The accompanying model passes 32 tests, including 12,675 concrete list/model comparisons and 4,374 affine pair/image checks. Those executions test the model; the generic proofs here remain written arguments, and the Lean implementation and usability questions remain explicit engineering experiments.

Contents

1 The proposal in one decision

1.1 Extend the type discipline at the point of use

Rowan's first implementation should extend *what Lean's elaborator can infer and explain from an object's specifications*, not the kernel's logical foundations. A positive real number should remain the same real number when an operation needs its nonzeroness. A function continuous on a region should retain that region when it is used in a composition. A synthesized program should be accepted under a contract about its actual behavior, rather than merely because its input and output types match.

The goal is a mathematical proof that records choices and arguments, while routine premise matching becomes inspectable generated evidence. This is not "Lean cannot express positivity" or "dependent types cannot express behavior." Lean already expresses both; its elaborator produces explicit terms for the kernel to check [10]. The research question is whether a coherent use-site interface can reduce author effort without hiding changes of meaning.

For example, the following is *proposed Rowan notation*, not currently accepted Lean or the full grammar of the supplied model:

```
operation exact_quotient(n, d : Int)
  requires d != 0 and d divides n
  returns q : Int ensuring d * q = n

given b : Int with even b
and p : Nat with 4 <= p
let q := exact_quotient(-(a^p) * b^p, 16)
```

Here `a` is an already declared integer. The operation leaves `a`, `b`, and `p` unchanged, constructs the uniquely specified integer `q`, and requests a divisibility proof. It does not reinterpret integer division as field division or silently add a Fermat equation to the hypotheses. A missing divisibility argument is a displayed obligation, not an automatically assumed property.

1.2 What is specifically Rowan's fourth-round increment?

The design keeps the reports' evidence overlays, dependent contracts, relation-specific observations, and ordinary Lean export. Its increment is a set of operational commitments that can be independently implemented.

A fixed meaning for each request. An operation receives an immutable, fully elaborated target in a typed local context. Search works in a separate candidate context. It may invent a witness for an authorized existential problem, but it cannot instantiate a parameter of the original statement to make that statement easier.

A finite generation policy. Automatic rules range over a finite pool of already elaborated terms and explicitly authorized operation outputs. Rowan bounds the number of theorem instantiations before computing closure. It consequently promises completeness only for a precisely identified finite rule problem, not for mathematical entailment in general.

An admissibility-aware behavioral boundary. An observation such as length has an image, and the input contract can further restrict that image. Rowan supplies a complete equality criterion for affine models on a supplied semilinear image, together with explicit counterexample coordinates. The image coverage and realization theorems are separate obligations of the source adapter. This is a

proposed integration of elementary mathematics, not a claim that affine algebra or semilinear sets are newly invented.

These commitments have a common purpose: a mathematical step must preserve *which claim is being made, over which objects, and for which possible inputs.* A concise surface is useful only after those questions have stable answers.

1.3 The deliverable and its limits

The package contains this self-contained article, its PDF, a standard-library Python model, tests, execution results, and a source register. The model parses three commands, generates sorted ground rules, computes relevant closure, replays symbolic derivations, reports missing premises, normalizes a small list-expression language, and checks affine equality on semilinear images. It is deliberately smaller than the proposed language.

Its symbolic registry is an *assumed theory of the finite model*, not a collection of kernel-checked Lean theorems. Its exact integer computation is not itself Lean evidence. The article proves the intended generic mathematical facts on paper and identifies the additional denotation, reification, and implementation obligations needed for a real Lean integration. There is no claimed compression ratio, author-time reduction, or rebuilt upstream corpus.

2 What the two round-three reports settle

2.1 Read the reconciliation, not just the headlines

The close reading used both main TeX reports at Brainstorm commit `bf333390` (the full revision is recorded in Appendix B): *From Properties to Proof Obligations* and *Nine Refinements* [1, 2]. The second report includes a reconciliation with the first. Their different commitment matrices are editorial decompositions of the same nine inputs, not independent observations that should be pooled.

Both recommend implementation over further expansion of the notation. They agree that the local object must remain fixed, that structures are data rather than inferred propositions, that behavioral composition needs the actual intermediate value, and that a checked final theorem does not by itself establish the fidelity of the authored statement or of a required proof method. Rowan adopts those constraints without renaming them as new inventions.

Several corrected distinctions matter directly to the present design. A sound finite certificate can establish a universal theorem even when the checker is incomplete. Conversely, abstract failure needs realization before it is a source-level refutation. Equality of affine coefficients is complete over unrestricted natural length inputs, but not over `List Empty`, where only zero is realized. The list interpreter's soundness theorem and the coefficient criterion must not be conflated: the reviews report Lean evidence for the former, while the latter remains a written argument in that corpus. Rowan follows the detailed evidence accounting and reconciliation where older summary sentences are broader [1, 2].

2.2 Inheritance and decisions

Topic	Retained from round three	Rowan's concrete decision
Object properties	Evidence about the same elaborated term.	Local facts stay in the Lean context; an index is a disposable lookup aid.
Function behavior	Quantified relational contracts, with dependent composition.	Distinguish guarded total functions, refined-domain functions, and authorized witness construction in the surface.
Inference	Backward requirements plus finite grounded closure.	Enumerate sorted assignments from a frozen finite term pool; give an explicit generation bound.
Meaning and scope	Freeze structures and statement parameters; respect branch contexts.	Seal a metavariable-free target relative to its telescope, then use a private proof island.
Finite observations	Separate soundness, completeness, realization, and coverage.	Add an image-indexed affine criterion, including finite unions and correlated observations.
CAS and synthesis	Untrusted discovery; exact target reconstruction.	Require source denotation and image evidence in addition to candidate identity and operational receipts.
Evaluation	Shared-library and shared-service baselines.	Test the use-site layer before attributing any gain to a new language.

Trellis's bounded closure and Schist's backward demands, as analyzed in the syntheses, motivate the inference engine. Fiber and Obsidian motivate the satisfiable Frey helper. Moraine's list semantics, Karst's empty-type example, and Basalt's and Tephra's realization warnings motivate the behavioral checker. Basalt's relational diagrams and Moraine's naturality example prevent the type interface from degenerating into unary adjectives. These debts are to the proposals as studied through the two syntheses; this article does not claim a new independent full audit of all nine original packages.

2.3 Do not overread earlier executions

The reviews report that seven supplied Lean cores compile and contain 49 named theorems. Their axiom-query inventories cover different subsets, so those counts are not contradictory. Schist's small affine checker reaches an original arithmetic target; Moraine's interpreter links a fixed list grammar to its length semantics; neither is an implemented general Rowan frontend. The reviews also distinguish a deliberately failing final `mvcgen` probe from a separately checked positive-only file [1, 2].

Those are useful starting artifacts, not measurements of human-scale proof writing. Rowan's new executions are the Python tests in Section 17. All statements about prior Lean acceptance remain attributed to the reviews.

3 The source-level problems Rowan should actually solve

3.1 A local identity must remain local

ProveIt's `AutonomousODE.iteratedDeriv_eq_comp` is a particularly good example because the mathematical argument is short and its existing Lean implementation is already structured [3]. Let $U \subseteq \mathbb{R}$

be open, and suppose $W' = \phi \circ W$ on U. A family G_n and derivative values H_n satisfy, at the points $W(x)$ with $x \in U$,

$$G_0(W(x)) = W(x), \qquad G_{n+1}(W(x)) = H_n(W(x))\phi(W(x)),$$

with G_n differentiable there and derivative H_n. The conclusion is $W^{(n)}(x) = G_n(W(x))$ on U for every n.

The Lean proof explicitly upgrades the inductive equality on U to eventual equality near a chosen $x \in U$, then uses derivative congruence and the chain rule. This is not gratuitous syntax: equality only at x cannot be differentiated. The useful compression is to make "on this open region" a persistent part of the identity's interface, so that the neighborhood bridge is generated when demanded. The hypotheses on G_n must remain restricted to $W(U)$; replacing them with global smoothness would prove a different, stronger-hypothesis theorem. Section 9 gives the full Rowan trace.

3.2 An arithmetic representation change has a contract

The FLT source already uses a `FreyPackage` structure containing numbers and proofs of their properties. It therefore refutes the simplistic premise that Lean lacks property-rich objects [4]. Its integral and rational Weierstrass models nevertheless require a compatibility proof because

$$(b^p - 1 - a^p)/4, \qquad -a^p b^p/16$$

have different division semantics in $\mathbb{Z}$ and $\mathbb{Q}$.

The solution file proves the required divisibilities, casts the exact quotients, and uses coefficientwise equality [5]. The author's mathematical intent is "these are integral coefficients, so the rational model is their base change." Rowan should preserve the divisibility argument while reducing repeated routing of powers, congruences, and casts. The whole source module contains other arithmetic lemmas; its total size must not be reported as the size of this one compatibility proof.

3.3 A universal property does not manufacture extra finiteness

The tensor-family theorem in the FLT source constructs an algebra W with natural equivalences

$$\mathrm{Hom}_{A\text{-alg}}(W, T) \simeq \prod_{i \in I} \mathrm{Hom}_{A\text{-alg}}(H_i, T).$$

It assumes I finite *and* that every H_i is finite and free as an A-module, in addition to the algebra structures. It returns finite/free properties of W and the naturality law for the equivalences [6].

Both syntheses eventually identify the omission of the factor finite/free hypotheses in one proposal's prose reconstruction. For a singleton family, a representing object is isomorphic to its sole factor; $\mathbb{Q}[X]$ provides the immediate obstruction to inferring finite dimension from finite indexing. A useful type layer should make this omission visible at the construction's use site, not make an attractive universal-property paragraph look verified.

3.4 Leant has important authority boundaries already

At the directly retrievable Leant snapshot `3a40904be8a410d832d9ab6900b3d3e7b425eccb`, the `Verified` wrapper records acceptance by a supplied verification callback. The source explicitly says

that this is not automatically a behavioral proof, solver certificate, or kernel proof. Its nominal association and exact candidate projection are valuable provenance mechanisms [7].

The Length adapter similarly binds a candidate to a particular translated query but explicitly does not authorize source-level refutation or pruning from model replay alone. It supports a separate paired-spine query interface, so joint outputs should not be reduced to unrelated scalar results [8]. Rowan extends this boundary with semantic and admissibility proofs; it does not describe the existing caution as a bug.

The reconciled reports discuss a later Leant identity, `823259f7`. That identity could not be retrieved through the repository connector in this session; a commit lookup returned "No commit found." The direct source observations here are therefore pinned to the available `3a40904b` snapshot. The reports' claims about the later snapshot are not silently promoted to independently verified facts.

4 A small mathematical surface

4.1 Seven forms, not unrestricted natural language

Rowan should begin with controlled mathematical statements embedded in Lean's expression language. Its proposed authoring forms are:

Form	Meaning
`given x : A with P x`	A theorem parameter and an explicit hypothesis.
`let x := t with P x`	A fixed definition plus an obligation to prove the property.
`know P by ...`	A proved assertion, never an additional assumption.
`show P by ...`	A fixed target whose proof is requested.
`obtain x satisfying P x`	Authorized witness construction under a fixed specification.
`on U, ...`	Scope for a relation restricted to the explicitly given region.
`using method ...`	A method application with typed roles and visible unmet premises.

Quantifiers, induction, case splits, calculations, and definitions remain ordinary structured proof constructs. "By symmetry" must name, or resolve to, an actual symmetry and its action on the statement; "without loss of generality" needs a reduction theorem returning the normalized witness and its reconstruction argument. Such phrases are not universal escape hatches.

A `given` clause visibly changes the theorem's hypotheses. A `know` clause cannot become `given` after a timeout. An unresolved operation shows a conditional interface or a proof hole in a draft, but a completed publication cannot hide that condition.

4.2 The readable proof is a projection of checked assertion nodes

The underlying document is a typed tree: declarations introduce binders, assertion nodes carry propositions, method nodes point to premises, and construction nodes bind new data. The compact renderer may collapse generated proof terms, but it should still display the object, quantifiers, region, measure, result relation, and open conditions that determine the claim.

The expansion view answers "why is this legal?" For a quotient it shows its divisibility and denominator obligations. For differentiation it shows the neighborhood relation. For a synthesized function it shows the universal contract and the actual term to which its proof applies. The view should be available at the point of use, not require reconstructing a global log.

Unparsed prose remains commentary. A renderer cannot certify the semantics of arbitrary explanatory English merely because nearby assertions are checked. Every sentence displayed as a formal assertion must be linked to its own typed assertion node, including assertions unused in the final proof. Statement alignment still needs a tested frontend and semantic reading tests.

4.3 Required reasons and helpful hints have different meanings

A mathematical reason can be a search hint, or a constraint on the accepted argument. Rowan uses explicit modes:

```
show P hinting orbital_comparison
show Q requiring exact_quotient_transport
```

The first permits another proof. The second uses an accepted derivation interface for the named method and its allowed supporting rules. Merely finding the method's name in a proof term is insufficient: an irrelevant occurrence does not show that the method justified the result.

A required method is an operational proof discipline, not a claim that Rowan can infer a human author's intention from an arbitrary term. The first implementation should enforce it through a small method grammar or a support-restricted helper theorem. The final theorem can be true while this separate requirement is unmet.

5 Properties and behaviors as an author-facing type system

5.1 Local refinements preserve the carrier

Let Γ be a Lean context, with the relevant structure arguments already chosen. Rowan writes

$$\Gamma \vdash t : A \ \triangleright \rho, \qquad \rho = \{p_1 : P_1(t), \dots, p_k : P_k(t)\}.$$

This abbreviates ordinary judgments $t : A$ and $p_i : P_i(t)$ in that same context. It is not a second source of assumptions. Its key operations are

$$\frac{t : A \quad p : P(t)}{t : A \triangleright \{P(t)\}}, \qquad \frac{p : P(t) \quad h : P(t) \to Q(t)}{h(p) : Q(t)}.$$

Combining rows combines proofs about the *same* term. It does not intersect two independently selected witnesses. A registered implication performs subsumption by an explicit proof application; it does not change Lean's definitional equality.

An exported value can use the existing subtype $\{x : A \mid P(x)\}$ or a structure with named fields. Locally, repeated packing and unpacking is usually unnecessary. The ordinary term is retained, and the evidence index points to its hypotheses or derived proofs. Open rows are a presentation device for available facts, not an assertion that arbitrary principal refinements can be inferred.

5.2 Structures, indices, and universes are part of the meaning

A topology on A, a multiplication on a carrier, an action, and a measure are not interchangeable evidence about a single unspecified structure. They are data determining the proposition. Two displays

of "continuous f" may hide distinct domain topologies. Rowan keys its facts to fully elaborated propositions, including universe instantiations, dependent arguments, and structure dictionaries.

The initial implementation should allow lexical structure selection and then use Lean's ordinary instance machinery for the selected data. It should not turn every fact into a global instance. Where a library API expects `Fact P` or another proposition-valued class, a local bridge may install the already proved fact for the duration of the call. It does not search for a new meaning of the operation.

The source studies motivate measuring this distinction, but they do not show that a lexical context will eliminate generated FLT preambles. That remains a specific migration experiment, not a promised line-count saving.

5.3 Two function interfaces that must not be conflated

For a fixed total function $f : A \to B$, define

$$\mathsf{Beh}(f; P, Q) := \forall x : A,\ P(x) \to Q(x, f(x)), \qquad Q : A \to B \to \mathsf{Prop}.$$

This is an ordinary proposition about f. It says nothing about its behavior outside P and does not make f partial. By contrast,

$$g : (x : A) \to P(x) \to B \quad \text{or} \quad g : \{x : A \mid P(x)\} \to B$$

requires evidence of domain membership at application. Such a g does not automatically extend to all of A. An extension requires data or a theorem that supplies the missing cases; any noncomputable choice must remain subject to the project's execution policy.

Rowan's syntax should distinguish a guarded total function from a restricted domain. For example, imported real division remains Lean's total operation; a cancellation method has a nonzero precondition. A newly defined exact quotient operation can instead require divisibility in its input interface. Neither interpretation should be selected solely because it makes a goal easy.

5.4 Behavior includes relations between evaluations

Output refinements alone are inadequate for monotonicity, Lipschitz bounds, permutation preservation, inverse pairs, and naturality. For instance,

$$\forall x, y,\ x \le y \Longrightarrow f(x) \le f(y)$$

is a law about two evaluations. A sorting specification needs both order and permutation of the *actual* input; length preservation does not suffice. An exact sequence is a relation among maps and their images and kernels, not three unary tags on its vertices.

Rowan therefore permits arbitrary Lean propositions as behavioral clauses, including dependent and higher-order relations. The automatic fragment may recognize only a finite collection of theorem shapes. Expressibility and automatic solvability are intentionally different guarantees.

5.5 Consequence and composition

Proposition 5.1 (Contract consequence)**.** *Suppose* $\mathsf{Beh}(f; P, Q)$, $P'(x) \Rightarrow P(x)$, *and* $P'(x) \wedge Q(x, y) \Rightarrow Q'(x, y)$. *Then* $\mathsf{Beh}(f; P', Q')$.

Proof. For an input satisfying P', obtain P, apply the original contract at that same input, and use the postcondition implication. □

Thus a client may strengthen what it supplies and weaken what it requests. Viewed as implementation replacement, an offered implementation must demand no more and promise no less than the client's interface. These are the same variance directions, not competing conventions.

Proposition 5.2 (Composition at the retained intermediate value)**.** *Let* $f : A \to B$ *and* $g : B \to C$*, with* $\mathsf{Beh}(f; P, Q)$ *and* $\mathsf{Beh}(g; U, V)$*. Suppose*

$$P(x) \wedge Q(x, y) \Longrightarrow U(y),$$
$$P(x) \wedge Q(x, y) \wedge V(y, z) \Longrightarrow W(x, z).$$

Then $\mathsf{Beh}(g \circ f; P, W)$*.*

Proof. For x with $P(x)$, put $y = f(x)$. The first contract gives $Q(x, y)$, the first bridge gives $U(y)$, and the second contract gives $V(y, g(y))$. The second bridge yields $W(x, g(f(x)))$. □

For dependent outputs $B(x)$, retain the actual binding $y : B(x)$ and state the second operation in that extended telescope. Do not flatten the sequence into an unordered set of guard strings. A useful elaborator creates obligations in dependency order and exposes their dependencies in the frontier.

A requirement transformer Req_f may encode a registered *sufficient* route to a desired postcondition. Its soundness is $\mathsf{Req}_f(Q)(x) \Rightarrow Q(f(x))$; composition gives $\mathsf{Req}_f(\mathsf{Req}_g(Q))$ as a sufficient requirement for $g \circ f$. It need not be a weakest precondition. The relation to Hoare and Dijkstra interfaces is deliberate, but real stateful effects should reuse their own semantics, rather than call every proof obligation an effect [15, 12].

6 Sealed proof requests and statement fidelity

6.1 A target is not a search variable

A Rowan request is a record

$$\mathcal{R} = (E, \Gamma, T, \mathcal{M}, \mathcal{A}),$$

where E is a pinned environment, Γ a typed telescope, T the elaborated target, $\mathcal{M}$ the permitted method profile, and $\mathcal{A}$ the axiom policy. At sealing, T and the types and values of its local parameters contain no unresolved metavariables. They may contain the fixed free variables of Γ. Lean's ordinary proof goals live in the proposed proof term, not as unspecified parameters of T.

This stronger initial rule avoids an ambiguous distinction between a harmless proof metavariable inside a statement and a meaning-bearing one. A future implementation may relax it through a separately justified interface, but it should not begin with heuristic classifications of statement holes.

An existential theorem $T = \exists y : B,\ R(a, y)$ is already a fixed target. Search is allowed to construct its witness. Similarly, a synthesis request for a term in $\{f : A \to B \mid \mathsf{Spec}(f)\}$ fixes the specification while authorizing the choice of f. Candidate-private data metavariables are therefore allowed. What is prohibited is assigning a parameter of a, changing R, changing the domain, or selecting a different hidden structure in the sealed request.

6.2 Proof islands as an implementation boundary

The intended elaboration transaction has three phases.

Resolve. Parse the assertion, elaborate mathematical expressions, resolve designated structures and notation, and form the fixed request. Show any unresolved meaning-bearing ambiguity here. A document may remain a draft; it may not send an ambiguous theorem to search and accept whichever reading succeeds.

Discharge. Give the producer a private elaboration state with the fixed context as a read-only boundary. It may create candidate-local metavariables, invoke tactics, construct witnesses, or ask external services for certificates. No producer API should return an edited original target as a substitute for its evidence.

Check and commit. Reconstruct the exact proposed term in the original context, check it against the sealed target, reject unresolved goals, inspect the selected axiom policy, and validate any required method derivation. Only then insert the result as a local fact or committed declaration.

In Lean metaprogramming, this calls for private snapshots and validation of which previously existing metavariables were assigned, together with a final check against the original expression. State rollback by itself is not a semantic proof, and a hash of the request is only a routing check. A retained term at T is the mathematical authority.

Proposition 6.1 (Meaning noninterference for the request interface)**.** *Consider an abstract request machine whose request record is immutable, whose producer can write only candidate-private state, and whose sole commit rule accepts a term checked at the original T in the original Γ. Every successful commit is associated with that same (Γ, T), independently of the producer's search path.*

Proof. Neither a producer transition nor its return value changes the immutable request. The only transition that publishes an assertion reads its target from that request, not from producer-supplied target metadata. Therefore each published assertion has the original context and target. The premise that the checker accepts the returned term supplies its typing at that target. □

This modest theorem specifies an API invariant, not the correctness of an arbitrary Lean plugin. It is stronger than a convention to "try not to change meaning," but it says nothing about whether the frontend originally parsed the mathematician's words as intended. That separate boundary requires a specified source semantics and validated source-to-request correspondence.

6.3 Three acceptances, not one green light

A completed node carries three independent statuses.

Check	Question	Characteristic failure
Theorem validity	Does the term prove T under the selected assumptions and axiom policy?	An unsolved premise or an unauthorized axiom.
Statement fidelity	Is this T the intended authored assertion?	The wrong branch, measure, domain, quantifier order, or result strength.
Method fidelity	Does the accepted derivation satisfy an explicitly required route?	A contradiction shortcut instead of the specified arithmetic argument.

This separation also improves diagnostics. A service can return a correct identity about a different reified expression; that is not mathematical nonsense, but it is not an answer to the request. A method

can fail while the theorem is true. A proof can be valid while a polished surrounding sentence incorrectly calls an upper bound optimal.

6.4 Support restrictions include data that carry assumptions

For an arithmetic helper, limiting visible hypothesis names is insufficient. A variable of type `FreyPackage` carries hypotheses through its fields, including the Fermat equation. A called lemma can hide further dependencies. The support restriction must concern the typed telescope and the permitted transitive theorem environment, not just names occurring in a surface script.

The practical first mechanism is to formulate the helper over plain integers and a natural exponent, with exactly the displayed arithmetic assumptions. A separate adapter specializes it to a Frey package. A required arithmetic route can additionally use a small checked derivation grammar. Contradiction is not prohibited globally: proof by contradiction is ordinary mathematics. It is excluded only from a benchmark or method profile intended to exercise a different argument.

There is no general automatic promise to find a logically minimal support or to determine that each assumption is necessary. The retained proof has an inspectable dependency slice, and a benchmark has an explicit inhabitant of its intended assumptions. Those are useful, testable claims. They are weaker than semantic minimality and much easier to enforce reliably.

7 A bounded inference algorithm, including rule generation

7.1 The finite automatic fragment

Let K be a finite pool of well-typed elaborated terms, partitioned by their permitted types or sorts. In a first implementation it consists of terms from the current statement and local context, their selected typed subterms, and outputs explicitly named by the current operation. A registered rule is a Lean theorem whose use-site shape is

$$\forall \vec{x},\ P_1(\vec{x}) \to \cdots \to P_m(\vec{x}) \to Q(\vec{x}),$$

with a validated telescope and a finite instantiation policy. A ground instance is accepted only when every data argument is selected from K and the resulting theorem application is well-typed. Its premises and conclusion are exact elaborated propositions.

No rule is allowed to enlarge K merely by creating a new compound term. Otherwise an innocuous schema could generate $f(x), f(f(x)), \ldots$ forever. A new explicitly authored operation may enlarge the pool once with its specified terms and trigger a new bounded problem. This is a staged authoring workflow, not unrestricted global saturation.

The reference implementation has a deliberately simpler sorted first-order version. Its terms carry a name, sort, meaning identity, and snapshot identity. It validates predicate signatures and enumerates all sorted assignments. It does not implement dependent Lean type checking, universe inference, reduction, or theorem registration from Lean declarations.

7.2 Generation has an explicit bound

For a schema r with k_r data variables, let $m_{r,i}$ bound the number of admissible terms for its ith variable after earlier telescope arguments have been fixed. Then the number of attempted assignments is at most

$$G \le \sum_r \prod_{i=1}^{k_r} m_{r,i}. \tag{1}$$

If every pool has at most M terms and each schema at most k variables, $G \le |\mathcal{R}|M^k$. Nullary schemas contribute one assignment. Failed type checks count toward the budget; they do not generate a usable rule.

For dependent telescopes, enumerate left to right and check each selected argument against its instantiated type. Universe choices and structure parameters must likewise come from specified finite choices or be fixed in the request. An unconstrained universe search is not covered by (1). Host elaboration costs and timeouts must be recorded separately from the finite combinatorial bound.

At this stage a request can stop as *unsupported by the profile* or *budget exhausted*. Neither is a negation. If generation completes, let $\mathcal{G}$ be the resulting finite set of ground rules and V all their premise and conclusion atoms, together with the initial facts and target. Only now is the ordinary closure theorem applicable.

7.3 Backward relevance, then forward closure

Starting at the target, collect all ground rules with that conclusion, then recursively collect their premises and alternative producers. This produces a finite backward cone. It does not assume that one sufficient route is the only mathematically possible route. It merely excludes ground rules that cannot participate in a derivation of the requested atom in this profile.

Within the cone, maintain a counter for the distinct missing premises of each rule. Add visible admissible hypotheses to a queue. Each newly proved atom reduces the counters of the rules that consume it. When a counter reaches zero, create a derivation node by applying that rule to its already available premise proofs. Zero-premise rules are initialized explicitly. An atom enters the queue only once; alternative proofs can be ignored in the simplest policy.

With L the total number of distinct premise occurrences in the retained rules, the worklist bookkeeping is $O(|V| + |\mathcal{G}| + L)$ using suitable indices. This excludes instantiation, expression comparison, host type checking, certificate checking, and proof-term size. In particular, a small number of inference nodes need not imply small Lean terms if subterms are repeated without sharing.

Theorem 7.1 (Finite closure, proof production, and relative completeness)**.** *Fix the completed finite ground problem, its admitted initial facts, and its allowed rules. The worklist on the backward cone terminates. Every derived atom has an acyclic derivation from those facts and rules. The resulting set is the least set of cone atoms containing the initial facts in the cone and closed under the retained rules. It is independent of fair processing order. The target is in this set exactly when it has a finite Horn derivation in the original completed ground problem under the same admission policy.*

Proof. Only finitely many atoms can be inserted, and every insertion is permanent. Counters decrease only when a previously absent premise becomes known, so there are finitely many queue and counter events. Each new derivation points only to proofs already constructed; this gives an acyclic graph and proves soundness relative to the rule theory by induction on insertion time.

The result contains the admitted initial facts in the cone. If all premises of a retained rule are present, its counter eventually becomes zero, so its conclusion is present; hence the result is closed. Conversely, every insertion belongs to any cone-closed set containing those initial facts, by induction on insertion time. It is thus the least such set. Induction on a finite derivation using the retained rules puts its conclusion in that set. Finally, every rule that can produce the target is in the backward cone, and the same holds recursively for every premise producer in a finite derivation of the target. Thus a target derivation in the original admitted problem is entirely retained. These arguments determine the resulting set, not a unique selected proof or a unique explanation. □

A cycle with no independently established premise produces no fact. A cycle reached from a seed may contribute ordinary consequences. There is no rule that converts a pending condition into an assumption solely because that condition would make another step succeed.

7.4 The frontier is explanatory, not a weakest-precondition oracle

If closure does not contain the target, Rowan reports missing propositions, the context in which they are needed, the operation consuming them, and registered sufficient routes. The model reports unresolved leaves of the backward cone; an unseeded cycle without such a leaf retains the target itself as an open item. This is a valid nonminimal explanation, not a smallest unsatisfiable core or a proof that every listed premise is necessary.

For the Frey example, a request for the fourth coefficient reaches only evenness, the exponent bound, an intermediate power-divisibility fact, and the final exactness atom. A request for the second coefficient also reaches oddness and the residue of a. This already prevents a broad property package from forcing unrelated premises onto every consumer.

An ungrounded analytic theorem can still be used by an explicit Lean tactic or method adapter outside the finite profile. Its returned proof is checked at the same sealed target. Rowan deliberately permits a predictable small automatic fragment inside an expressive ambient proof system.

7.5 How property automation differs from type-class search

The engine is not intended to replace Lean's instance search or `fun_prop`. Mathlib already decomposes functions compositionally to prove properties such as continuity and differentiability, consults local facts, and exposes side-condition dischargers [11]. Such machinery should be a provider for appropriate Rowan obligations and a control in evaluation.

The proposed added value is uniform treatment of use-site demands across several relation kinds: an exact quotient, a locally valid identity, an admissible observation, and a dependent construction can share the same source-to-obligation interface. The finite registry provides transparent composition and failure reporting where a direct existing tactic is not already sufficient. If it only duplicates cheap existing lookup, the extra layer should be removed.

8 Rewriting, scope, and observation-specific transport

8.1 A proof can outlive a printed expression, but not its context

Suppose a known proof has type $P(t)$ and simplification changes a displayed expression to u. There are three distinct cases. If Lean accepts the necessary conversion, the proof can be reused directly. If a checked equality $e : t = u$ is available, equality elimination yields the transported proof at u. If only $P(t) \leftrightarrow Q(u)$ is established, use its forward implication without asserting that $t = u$.

A pretty-printed normal form is not an equality proof. Even matching syntax can hide different structure arguments. Dependent changes are especially important: when $v : B(t)$ is transported along $t = u$, the predicate on v may also depend on t. The implementation must check the complete transported term and target, rather than update one anchor field in a cache.

Rowan's default index lifetime is one elaboration snapshot. After an upstream edit, rebuild it from the new local context. Reusing old entries is an optimization available only through a checked context substitution mapping old variables and hypotheses to terms and proofs in the new context. A binder rename might admit such a substitution; a change of measure usually does not preserve an integrability assertion. The hash of a receipt or the reuse of an old local identifier establishes neither fact.

8.2 Relations name what the next operation may observe

For objects $a : A$ and $b : B$, a result package is

$$\mathsf{Result}(a, R) = \{b : B \mid R(a, b)\}.$$

The relation may be exact equality after interpretation, equality on a region, equality almost everywhere for a fixed measure, an enclosure, or finite-jet agreement. A consumer C registers the relation it respects:

$$R(a, b) \Longrightarrow S(C(a), C(b)),$$

with any guards stated in the actual theorem. These relations are not a single confidence ladder.

For instance, if integrable real-valued functions agree almost everywhere with respect to μ, their integrals are equal. The almost-everywhere relation can therefore feed that integral consumer. It cannot in general feed point evaluation: functions differing at one point under Lebesgue measure have equal integrals but can have different values there. In a concrete Lean adapter, use the exact signature of the available integral-congruence theorem; do not add integrability to that lemma's premises merely because an adjacent operation needs it.

Likewise, for each $a \in \mathbb{R}$, the function $x \mapsto \mathbf{1}_{\{a\}}(x)$ is zero almost everywhere under Lebesgue measure. There is nevertheless no point at which all these functions vanish simultaneously: choose $a = x$. A countable intersection of full-measure events is available through the corresponding theorem; an arbitrary uncountable one is not. An evidence row must retain the quantifier order and the indexing hypotheses.

8.3 Precision is an input demand, not a cosmetic annotation

For formal series over a commutative coefficient ring, let $f \equiv_N g$ mean equality of coefficients of degrees less than N. Then

$$f \equiv_{N+1} g \Longrightarrow f' \equiv_N g'.$$

The proof is coefficientwise: the coefficient of degree $k < N$ in a derivative uses the original coefficient of degree $k+1 < N+1$. A proposed method that retains the same order after differentiation has failed its contract. At order N itself the pair X^N and 0 illustrates the problem over, for example, $\mathbb{Q}$ with $N > 0$.

If f requires precision $d_f(N)$ to produce precision N, and g requires $d_g(N)$, the pipeline $g \circ f$ has the sufficient demand $d_f(d_g(N))$. The formula assumes the input and output observation interfaces actually compose. It cannot turn finitely many verified prefixes into an infinite exact identity. That conclusion requires a theorem quantified over all observations and a separation argument.

8.4 New witnesses are not new views of the old witness

Dividing a triple by its gcd, changing coordinates, selecting a prime divisor of an exponent, or choosing a basis produces data. The correct judgment is a construction with a relation between old and new data, not an equality-based refinement of the old object. Rowan's `obtain` form therefore returns a witness together with its preservation or reconstruction theorem.

This matters in the tensor example: choosing W and its structure is an authorized existential construction. Its representing equivalence, naturality, and finite/free properties are distinct obligations about that chosen W. A proof of existence of some finite type does not itself provide an executable enumeration; the computational interface must request suitable data or accept an explicitly noncomputable choice.

9 Worked development I: differentiate a local identity

9.1 The exact theorem to preserve

The following is the mathematical form of the relevant ProveIt interface, with the derivative-value family renamed H for readability [3].

Theorem 9.1 (Autonomous derivative family)**.** *Let $U \subseteq \mathbb{R}$ be open. Let $W, \phi : \mathbb{R} \to \mathbb{R}$ and $G, H : \mathbb{N} \to (\mathbb{R} \to \mathbb{R})$. Suppose that, for every $x \in U$ and every n,*

$$W'(x) = \phi(W(x)), \qquad \text{with a derivative at } x, \tag{2}$$

$$G_0(W(x)) = W(x), \tag{3}$$

$$G_n'(W(x)) = H_n(W(x)), \qquad \text{with a derivative at } W(x), \tag{4}$$

$$G_{n+1}(W(x)) = H_n(W(x))\phi(W(x)). \tag{5}$$

Then, for every $n \in \mathbb{N}$ and every $x \in U$,

$$W^{(n)}(x) = G_n(W(x)).$$

Here the iterated derivatives have the meaning of Lean's iterated derivative operator, and the derivative hypotheses assert existence rather than merely an equality involving a totalized derivative value.

Proof. Induct on n with the statement universally quantified over $x \in U$. For $n = 0$, the claim is (3). Suppose it holds for n and fix $x \in U$. Openness supplies a neighborhood of x contained in U. The functions $W^{(n)}$ and $G_n \circ W$ therefore agree on a neighborhood of x. By (2) and (4), the chain rule

gives a derivative of $G_n \circ W$ at x, with value $H_n(W(x))\phi(W(x))$. Neighborhood equality transfers that derivative to $W^{(n)}$. Hence

$$W^{(n+1)}(x) = H_n(W(x))\phi(W(x)) = G_{n+1}(W(x)),$$

where the last equality is (5). Since x was arbitrary, the inductive statement holds throughout U. □

The argument does not require G_n differentiable away from $W(U)$, nor a separate global smoothness assumption on ϕ. Those would be unnecessary changes to the source theorem. As a satisfiable mathematical fixture, take $U = \mathbb{R}$, $W(x) = e^x$, $\phi(w) = w$, $G_n(w) = w$ for all n, and $H_n(w) = 1$.

9.2 The intended authoring trace

Assuming the hypotheses have been introduced with their exact regions and quantifiers, the proof could be written:

```
show for every n, D[n] W = G[n] o W on U by induction n
  zero:
    use the initial identity on U
  next n, identity_on_U:
    fix x in U
    know D[n] W = G[n] o W near x using openness of U
    differentiate this local identity using the chain rule
    finish using the recurrence at (n, W(x))
```

This is a proposed surface for the already explained proof, not an implemented parser feature. "Near x" denotes an eventual-equality relation for the neighborhood filter, not a numerical closeness test. "Differentiate" is a method with the two derivative premises and a neighborhood-congruence bridge. The region and the induction scope are part of the typed assertion nodes.

A bad elaboration would fix x *before* forming the inductive hypothesis, then try to differentiate an equality available only at that point. Rowan should show the missing neighborhood equality, not search for unrelated algebraic facts. The invalid neighbor $f(t) = t$, $g(t) = 0$ at $x = 0$ has $f(0) = g(0)$ but $f'(0) \neq g'(0)$.

The novelty here is not a new analysis lemma: the original Lean proof already uses the correct bridge. Rowan's proposed gain is making the bridge a reusable consumer contract, while preserving the precise theorem. An ordinary Lean wrapper that achieves the same authoring benefit is an essential baseline.

10 Worked development II: exact Frey coefficients

10.1 A satisfiable arithmetic context

Use integers a, b and a natural p satisfying

$$\mathcal{F}_0 : \qquad p \geq 4, \quad p \text{ odd}, \quad a \equiv 3 \pmod 4, \quad 2 \mid b. \tag{6}$$

No primality, coprimality, nonzeroness, variable c, or Fermat equation is needed. The triple $(a, b, p) = (3, 2, 5)$ satisfies these assumptions. This is the support-restricted arithmetic benchmark emphasized in the reconciled reports [1, 2] and extracted from the upstream transport proof [5].

Proposition 10.1 (Operation-specific divisibility)**.** *Under* (6)*,*

$$4 \mid b^p - 1 - a^p, \qquad 16 \mid -a^p b^p.$$

The second conclusion uses only $2 \mid b$ *and* $p \geq 4$*.*

Proof. Write $b = 2k$. Since $p \geq 4$, 2^p is divisible by both 4 and 16; hence so is b^p. Since $a \equiv -1 \pmod 4$ and p is odd, $a^p \equiv -1 \pmod 4$. Thus $b^p - 1 - a^p \equiv 0 \pmod 4$. Finally, $16 \mid b^p$ implies $16 \mid -a^p b^p$ for every integer a. □

For the first conclusion, the power bound can in fact be weakened to $p \geq 2$ while keeping the other assumptions. Rowan's initial three-rule model uses the common $p \geq 4$ context for simplicity. Its frontier describes that registered sufficient route, not a mathematically weakest set of assumptions.

10.2 An exact quotient has a balance law

Define

$$n_2 = b^p - 1 - a^p, \quad n_4 = -a^p b^p, \qquad q_2 = n_2/4 \in \mathbb{Z}, \quad q_4 = n_4/16 \in \mathbb{Z},$$

where the quotient is the original integer division. The divisibilities give

$$4q_2 = n_2, \qquad 16q_4 = n_4.$$

For any $n, d \in \mathbb{Z}$ with $d \neq 0$ and $d \mid n$, these balance equations also characterize the quotient uniquely. The operation's public result may therefore be the integer value together with the balance proof. Packaging that proof as a property does not change the quotient.

Proposition 10.2 (Transport balance before inversion)**.** *Let* $j : \mathbb{Z} \to R$ *be a unital ring homomorphism to a commutative ring. If* $dq = n$*, then* $j(d)j(q) = j(n)$*. If* u *is a unit of* R *with value* $j(d)$*, then*

$$j(q) = u^{-1} j(n),$$

where the displayed inverse is the value of the inverse unit. In a field, if $j(d) \neq 0$*, this is* $j(q) = j(n)/j(d)$*.*

Proof. Apply j to $dq = n$ and use preservation of multiplication. Multiplication by u^{-1} gives the second equality; the field case identifies it with ordinary division by the nonzero denominator. □

This formulation distinguishes three facts that should not be collapsed into the adjective "invertible enough": nonzero, multiplicatively regular, and a unit. The integer 2 is nonzero and regular in $\mathbb{Z}$ but not a unit. Under the map to $\mathbb{Z}/2\mathbb{Z}$ it becomes zero. Thus the source fact $d \neq 0$ alone cannot justify inversion after a change of coefficient ring.

For $R = \mathbb{Q}$, both denominator images are nonzero, so

$$(q_2 : \mathbb{Q}) = \frac{b^p - 1 - a^p}{4}, \qquad (q_4 : \mathbb{Q}) = -\frac{a^p b^p}{16}.$$

The rational expressions on the right use rational casts of a, b. Extensionality of the five Weierstrass coefficients then proves the integral model's base change equals the rational model. The three remaining coefficients are the matching constants $1, 0, 0$. Coefficient casts alone are not silently treated as equality of the whole record; the record extensionality step is explicit.

10.3 The Rowan use site and its generated evidence

```
given a, b : Int
and p : Nat with odd p and 4 <= p
and a == 3 mod 4 and even b

let a2 := exact_quotient(b^p - 1 - a^p, 4)
let a4 := exact_quotient(-(a^p) * b^p, 16)
know the rational coefficients are casts of a2 and a4
  requiring exact_quotient_transport
show base_change(integral_model, Rational) = rational_model
  by equality of coefficients
```

The compact proof is short because the arithmetic package and the representation-change method have useful interfaces, not because the cast obligations have ceased to exist. Their generated proofs and support remain inspectable. An existing ordinary Lean helper can express exactly the same mathematics and must be available to the baseline arm.

In the executable model, there are two integer objects and one natural object. Sorted grounding produces ten instances of three schemas. The fourth coefficient's backward cone retains two rules and derives its exactness from just `even_b` and `bound_p`. Before oddness and the residue of a are introduced, the second coefficient is open with precisely those two missing leaf propositions. These are actual model outputs; the schemas are symbolic assumptions, not proofs freshly checked in Lean.

10.4 Adversarial neighbors

Input (a, b, p)	Removed premise	Concrete obstruction
$(3, 2, 5)$	None	$q_2 = -53$, $q_4 = -486$; a satisfiable positive fixture.
$(3, 2, 4)$	Odd exponent	$n_2 = -66$, not divisible by 4.
$(3, 3, 5)$	Even b	$n_2 = -1$, not divisible by 4.
$(3, 2, 3)$	$p \geq 4$	$n_4 = -216$, not divisible by 16.
$(1, 2, 5)$	Residue of a	$n_2 = 30$, not divisible by 4.

The checker must also reject transporting the integer quotient $1/4 = 0$ as the rational number $1/4$. These explicit obstructions show why the missing guards matter. In contrast, failure to find a divisibility proof for an arbitrary new input is merely unresolved; it is not automatically one of these counterexamples.

11 Admissibility-indexed behavioral types

11.1 Observe possible inputs, not an arbitrary ambient space

Let A be a concrete input type, $P : A \to \mathsf{Prop}$ its admissibility predicate, and $o : A \to U$ an observation. For lengths, $U = \mathbb{N}^m$. An abstract set $S \subseteq U$ can carry two independent interfaces:

$$\mathsf{Cover}(P, o, S): \quad \forall x,\ P(x) \Longrightarrow o(x) \in S, \tag{7}$$

$$\mathsf{Realize}(P, o, S): \quad \forall u \in S,\ \exists x,\ P(x) \wedge o(x) = u. \tag{8}$$

Together they state that S is exactly the admissible observation image. Coverage alone permits a sound overapproximation. For a specific negative witness, realization of that witness is enough; global surjectivity is not required.

The proposed behavioral type is therefore indexed not merely by an output formula, but also by this source observation interface. The notation

$$f : (A, P) \xrightarrow[o,S]{M_f} (B, r)$$

means that f is a fixed program, $r : B \to V$ observes its output, and a proved model law relates $r(f(x))$ to $M_f(o(x))$ on admissible inputs. The arrows are author-facing specification notation. Their Lean meaning is a collection of ordinary functions and propositions.

11.2 Positive and negative transfer require different directions

Suppose a source contract is $C_f(x)$ and its abstract predicate is $\widehat{C}_f(u)$. A useful adapter proves, on admissible inputs,

$$C_f(x) \iff \widehat{C}_f(o(x)).$$

The equivalence is convenient but not always available or necessary. For a positive proof, the direction $\widehat{C}_f(o(x)) \Rightarrow C_f(x)$, together with coverage, suffices. For a negative witness, the direction $C_f(x) \Rightarrow \widehat{C}_f(o(x))$, together with realization of the failing observation, suffices.

Proposition 11.1 (Source transfer)**.** *Under coverage and positive transfer, a proof that $\forall u \in S, \widehat{C}_f(u)$ proves $\forall x, P(x) \Rightarrow C_f(x)$. Under negative transfer, a failing $u \in S$ and an admissible x with $o(x) = u$ refute that source universal contract.*

Proof. For the positive direction, apply coverage to each admissible x, instantiate the abstract universal theorem at $o(x)$, and use the transfer implication. For the negative direction, a true source universal contract would give $C_f(x)$ at the realized input and hence $\widehat{C}_f(u)$, contradicting the failing observation. □

An overapproximation can make a positive certificate harder to obtain without making it unsound. It can also produce impossible negative witnesses. These are complementary facts, not a reason to discard abstraction altogether.

11.3 A list grammar with a provable affine semantics

For m list inputs, consider expressions generated by input variables, the empty list, cons with a supplied element, append, reverse, and map by a fixed function. In the executable companion the element type is `Nat` and the only map is the concrete successor function; there is no assumed theorem about an arbitrary external provider.

Define an affine length form

$$M_e(n_1, \ldots, n_m) = c_e + \sum_{i=1}^{m} a_{e,i} n_i, \qquad c_e, a_{e,i} \in \mathbb{N}.$$

The normalization rules are

$$\begin{aligned} M_{\mathsf{nil}} &= 0, & M_{\mathsf{input}_i} &= n_i, \\ M_{\mathsf{cons}(a,e)} &= 1 + M_e, & M_{e_1 +\!+ e_2} &= M_{e_1} + M_{e_2}, \\ M_{\mathsf{reverse}(e)} &= M_e, & M_{\mathsf{map}(h,e)} &= M_e. \end{aligned}$$

A provider extension must establish the corresponding semantic rule about the exact provider function; a registration string is not that theorem.

Lemma 11.2 (Length denotation)**.** *For every well-typed expression in this grammar and every concrete input environment $\vec{x}$,*

$$\mathrm{length}([\![e]\!](\vec{x})) = M_e(\mathrm{length}(x_1), \ldots, \mathrm{length}(x_m)).$$

Proof. Induct on the expression. The input and empty-list cases are immediate. Cons increases the length by one. Append adds the two lengths. Reverse and map preserve length. Substitution of the induction hypotheses gives precisely the stated affine normalization rules. □

The lemma links an interpreter to a model, not an arbitrary pretty-printed program to that interpreter. A Leant adapter additionally needs a checked connection between its selected candidate, the reified expression, and the concrete interpretation. The candidate's exact spelling should be replayed; a different spelling or an incomplete proof state cannot stand in for it.

11.4 The empty-type case is an image issue

For an arbitrary type A,

$$\{\mathrm{length}(xs) : xs : \mathsf{List}(A)\} = \{n \in \mathbb{N} : n = 0 \ \vee \ \mathsf{Nonempty}(A)\}. \tag{9}$$

At $n = 0$, use the empty list. A nonempty list supplies an element of A. Conversely, a supplied element can be repeated to realize any positive length.

For $A = \mathsf{Empty}$, the image is $\{0\}$. The identity program and the constant-empty program have affine length models n and 0, but their lengths agree on every concrete input. Coefficient inequality on the ambient $\mathbb{N}$ is therefore not a source refutation. For $A = \mathsf{Nat}$, length one is realized by $[0]$, which is a valid counterexample to the same proposed length-preservation law for the constant-empty program.

Equation (9) is a mathematical statement about `Nonempty A`. Producing an executable realizer requires an actual element or an explicitly permitted choice mechanism. Proof of nonemptiness in `Prop` should not be silently advertised as executable enumeration. Moreover, the grammar over an empty element type cannot contain a cons literal without element data. A length model is not a license to construct an ill-typed source program.

11.5 Correlated observations are not a Cartesian product

Observe one list by

$$o(xs) = (\mathrm{length}(xs), \mathrm{length}(\mathsf{reverse}(xs))).$$

Over natural lists its image is the diagonal $\{(n, n) : n \in \mathbb{N}\}$, not $\mathbb{N}^2$. The affine forms n_1 and n_2 are equal on that diagonal although their coefficient vectors differ. The ambient witness $(1, 0)$ does not describe any input to this observation.

By contrast, for two independent list inputs the image really is $\mathbb{N}^2$. The same two affine forms are then inequivalent, with $([0], [])$ as a concrete witness. A paired-output adapter must preserve the whole observation configuration and its input dependencies. Marginal realizability of each coordinate is insufficient.

11.6 A complete affine criterion on supplied semilinear images

Definition 11.3 (Finite union of linear sets)**.** A supplied semilinear image in $\mathbb{N}^m$ is a finite union

$$S = \bigcup_{j=1}^{r} \left\{ b_j + \sum_{k=1}^{s_j} z_k v_{j,k} : z_k \in \mathbb{N} \right\}, \qquad b_j, v_{j,k} \in \mathbb{N}^m. \tag{10}$$

The empty union is allowed. A component with no generators is a singleton. The data need not be minimal, disjoint, or linearly independent.

Let $F, G : \mathbb{N}^m \to \mathbb{Z}$ be affine forms and write

$$D(n) = F(n) - G(n) = \delta + \alpha \cdot n, \qquad \delta \in \mathbb{Z}, \quad \alpha \in \mathbb{Z}^m.$$

The following theorem gives Rowan's proposed admissibility-aware checker.

Theorem 11.4 (Base-and-generator equality criterion)**.** *For the supplied set S in (10), the following are equivalent:*

$$\forall n \in S, \quad F(n) = G(n);$$
$$\forall j, \quad \delta + \alpha \cdot b_j = 0 \quad \textit{and} \quad \forall k,\ \alpha \cdot v_{j,k} = 0.$$

If the criterion fails, it constructs a failing point of S of the form b_j or $b_j + v_{j,k}$.

Proof. Suppose $F = G$ on S. Each base b_j belongs to its component, so $D(b_j) = \delta + \alpha \cdot b_j = 0$. Each $b_j + v_{j,k}$ also belongs to that component. Subtracting $D(b_j) = 0$ from $D(b_j + v_{j,k}) = 0$ gives $\alpha \cdot v_{j,k} = 0$.

Conversely, for any point in a component,

$$D\left(b_j + \sum_k z_k v_{j,k}\right) = \delta + \alpha \cdot b_j + \sum_k z_k (\alpha \cdot v_{j,k}) = 0.$$

Therefore the two forms agree on the union.

For a failing criterion, first test the bases. If $D(b_j) \neq 0$, return b_j. Otherwise choose a failing generator with $\alpha \cdot v_{j,k} \neq 0$. Then $D(b_j + v_{j,k}) = \alpha \cdot v_{j,k} \neq 0$, so return that point. Its membership witness is the all-zero generator tuple with a single 1 in position k. No general membership decision procedure is required. □

The test takes $r + \sum_j s_j$ exact scalar conditions, with m-dimensional dot products. Its arithmetic bit complexity depends on the integer sizes; calling it constant-time because the number of symbolic conditions is small would be misleading. For vector-valued affine observations, apply the same criterion to every row of the difference matrix.

Corollary 11.5 (Complete source length comparison on an exact image)**.** *Suppose two fixed source expressions have the proved length denotation above, and (7) and (8) establish that their common admissible input observation has the supplied image S. Then the criterion decides their universal length equality on admissible inputs. A failure, together with a realizer for its returned point, yields a concrete source counterexample.*

Proof. The denotation lemma equates source length comparison with affine comparison at the input observation. Coverage transfers a successful universal abstract comparison to the source. If the abstract comparison fails, realization and the same denotation lemma transfer the constructed witness to a source failure. These directions give both sound acceptance and decision completeness. □

11.7 A useful range without an unbounded promise

Several important images fit the supplied format directly:

Domain or observation	Linear description	Required source evidence
Independent natural-list lengths	Base 0, unit-vector generators.	Repeat 0 independently in each input.
Lengths over an empty type	Base 0, no generators.	Every input list is empty.
One list and its reverse	Base $(0, 0)$, generator $(1, 1)$.	Reverse preserves length; realize with a repeated element.
One even length	Base 0, generator 2.	A guard certifying even input length and its realization.
Paired lengths $(n + 2, n + 3)$	Base $(2, 3)$, generator $(1, 1)$.	The stated joint input construction.
A finite alternative of such cases	A finite union of components.	Coverage and realization for each permitted branch.

The theorem does not infer a semilinear image of arbitrary Lean code. It checks affine equality once a suitable image interface is supplied. A length operation such as a value-dependent filter can leave the affine fragment. A fixed drop operation may admit a piecewise-affine extension, but its branch partition and denotation need additional proofs. More general QF_LIA contracts, inequalities, recursion, and provider behavior do not inherit completeness merely because Leant has an SMT translation.

Likewise, passing a length contract does not prove sorting, stability, or permutation. These need their own source properties or sound abstractions. The point of a rich behavioral type is to retain such distinctions, not to replace them all by the easiest observable quantity.

11.8 A candidate is not a sketch

The construction above refutes a *fixed* candidate or a fixed pair of expressions. A sketch permits a family of completions. One failed completion says nothing about a different completion. Sound rejection of an entire sketch requires a theorem that every permitted completion violates the contract, or a necessary-condition theorem whose failure covers that whole family. Heuristic pruning may be useful, but it must remain labeled heuristic and must not be exported as a mathematical impossibility result.

12 A proof assistant and a certified CAS, without a blurred boundary

12.1 Two computational routes, one mathematical interface

Rowan should allow both verified algorithms and untrusted discovery. In the first route, an algorithm implemented in Lean is accompanied by a correctness theorem; invoking it and proving its premises produces the result. In the second, an external CAS or synthesis engine proposes a result and a

certificate, and a small verified checker reconstructs the required evidence. The systematic separation of discovery and checking has a long precedent in skeptical CAS integration [16]. Rowan's addition is to expose the result's exact mathematical relation and its input demands at the use site.

A service interface should consequently name the source expression, its interpretation, the proposed result, and the promised relation. A polynomial normalizer, root isolator, integration method, and numerical enclosure provider have different guarantees. They should not all return an unqualified "simplified answer."

12.2 A polynomial certificate has an original-target bridge

For an elementary exact fragment, take polynomials in d variables with integer coefficients. A sparse polynomial is a finite map $\mathbb{N}^d \to \mathbb{Z}$, with absent coefficients interpreted as zero. For a commutative ring R, the denotation at $v \in R^d$ is

$$[\![p]\!]_v = \sum_{\alpha} \iota(c_\alpha) \prod_{i=1}^{d} v_i^{\alpha_i},$$

where $\iota : \mathbb{Z} \to R$ is the canonical map. Finite dimensions, exponents, and the coefficient interpretation are part of the type or checked format.

Proposition 12.1 (Ideal-combination certificate)**.** *Suppose a certificate supplies $h_1, \ldots, h_k$ and exact polynomial normalization checks*

$$p = \sum_{i=1}^{k} h_i g_i.$$

Then in every commutative ring, at every valuation v with $[\![g_i]\!]_v = 0$ for each i, one has $[\![p]\!]_v = 0$.

Proof. Evaluation preserves addition and multiplication. Hence $[\![p]\!]_v = \sum_i [\![h_i]\!]_v [\![g_i]\!]_v = 0$. The preservation lemmas follow from the finite coefficient definitions of addition and convolution. Normalization combines equal exponent vectors and removes zero coefficients, so it preserves this denotation. □

A verified implementation still needs those normalization and convolution lemmas, a terminating executable checker, and a theorem about its actual return value. In addition, a reifier must prove that p and the g_i denote the *original* Lean expressions and assumptions. A correct identity in a different polynomial problem does not prove the original goal.

The format must check arity before combining arrays. A truncating zip can silently change the theorem by dropping an unmatched term or variable. Even better, use dependent vectors indexed by d and k so that a malformed certificate cannot enter the well-typed checking function.

Polynomial coefficient comparison is sound for identities interpreted in all commutative rings, but not a complete decision procedure for equality of polynomial *functions* on an arbitrary fixed ring. For example, $X^2 - X$ is a nonzero formal polynomial that vanishes at both elements of $\mathbf{F}_2$. Failed coefficient equality is therefore not automatically a counterexample to a source-level functional identity. This is the same admissibility lesson as the empty-list example in another domain.

12.3 One bridge interface, not one universal reifier

Several checkers can share the interface

$$e \rightsquigarrow \widehat{e} \quad \text{with proof} \quad [\![\widehat{e}]\!]_\eta = e,$$

where η supplies typed opaque atoms and their meanings. This permits an affine checker and a polynomial checker to reuse a ring-expression reifier where their grammars genuinely overlap. It does not make a list interpreter, a ring expression, and an analytic function the same grammar.

Rowan should separate the *contract* of reification from each implementation of it. A shared interface records the original term, environment, interpretation, and equality or other bridge theorem. The certificate service consumes that interface; it never receives authority to replace the original target with its own reification result.

The first useful Lean slice can use existing arithmetic tactics as providers. A custom sparse checker is justified by measurable benefits such as smaller certificates, predictable replay, or support for an external CAS. It is not required simply to claim a hybrid architecture.

12.4 Branches, completeness, and conditional answers

For square roots over $\mathbb{R}$, a certificate that $y^2 = x$ does not identify the nonnegative square root unless $y \geq 0$ is also established. Similarly, $\sqrt{x^2} = |x|$ can simplify to x only under a nonnegativity argument. A branch condition is part of the displayed result, not hidden search state.

For a solution set, Rowan distinguishes a certified witness, a sound list of solutions, a covering list, and an exact solution set. Exactness is

$$\forall x, \quad x \in S \iff P(x).$$

A list containing every solution and some non-solutions is not exact, just as an interval enclosure is not an exact value. A singleton enclosure may imply an exact value by a separate theorem; a finite jet may imply an exact polynomial identity if a degree bound is supplied. Promotion between result relations is always a theorem application with its own premises.

Symbolic integration should return the relation actually established: a primitive on a region, an almost-everywhere derivative identity, or a definite integral with convergence hypotheses. A numerical service can return rational lower and upper bounds plus a proof that the true quantity lies between them. No service should change an exact request into an approximation without an explicitly different result interface.

12.5 Actual program effects deserve actual specifications

A CAS implementation may use mutation, exceptions, or bounded search. Those are computational effects, not logical adjectives about a mathematical object. Lean's documented `mvcgen` workflow uses predicate-transformer semantics and specification lemmas to generate verification conditions for monadic programs [12]. Rowan should delegate appropriate program verification to such host mechanisms instead of building a second ad hoc Hoare logic.

A failed search returns an operational status, not a false mathematical proposition. A partial decision procedure may specify that every successful result is correct without claiming termination with an answer on all inputs. A total certified algorithm additionally needs its termination argument. Neither property follows from a list of successful test runs.

13 The Leant integration contract

13.1 Synthesis supplies evidence, not a different problem

Rowan sends Leant a fixed term type or a fixed specification-bearing type, the relevant local telescope, and operational limits. The system can search for a proof term, a suitable composition, or an explicitly authorized witness. A proposition-valued side condition must return a proof of that proposition; a function request must return the function *and* evidence of the fixed behavioral specification when behavior is part of the request.

The source-level connection should retain four things: the sealed original request; the exact accepted candidate; the bridge between that candidate and any abstract model; and the final proof or reconstructible certificate at the original target. A status word such as `Verified` cannot replace any of these. This is consistent with the direct Leant source's own distinction between callback acceptance and stronger evidence [7].

A tactic suggestion is another output form. It is considered checked only after replay of the exact displayed edit in the intended context. A different successful tactic invocation or an earlier candidate's receipt does not certify the edited text shown to the author. The author should be able to accept the suggestion and obtain the same mathematical assertion.

13.2 Reuse the existing provenance discipline

The retrieved `PostVerification` module uses a fresh abstract epoch to scope candidate handles, checks that a proposed ordering is a bounded permutation, and rejects omission, duplication, and reassociation [9]. Rowan should preserve that discipline. A ranking or behavioral assessor may reorder existing candidates; it may not borrow one candidate's authority for another candidate with a similar displayed type.

The Length adapter already separates query preparation, translation refusal, and model-level authority [8]. Rowan's image interface adds an explicit semantic layer above that boundary. A model witness remains model-relative until the exact source adapter proves the transfer direction and realizes the witness in the correct joint input domain.

13.3 A practical outcome vocabulary

Outcome	Permitted interpretation
Unsupported	The selected profile cannot represent this request.
Exhausted	The operational search or instantiation limit was reached.
No derivation in profile	Completed finite closure did not derive the target; ambient mathematical falsity is not implied.
Model counterexample	A precise abstract problem has a failing witness; source admissibility is not yet established.
Realized source witness	A concrete admissible input fails the fixed source contract; a Lean proof is still needed for kernel-certified publication.
Checked refutation	A retained Lean proof refutes the fixed claim under the chosen policy.
Checked evidence	A retained term proves the original target, with statement and required-method checks tracked separately.

Positive and negative output formats should be typed rather than inferred from wording. In the executable model the affine checker returns either agreement on the supplied image or a base/generator witness with component coordinates. It does not itself return "the Lean program is incorrect."

13.4 What should actually be measured

Leant's contribution should be measured on the obligations Rowan generates, not on hand-picked independent inhabitation examples. Separate direct lemma lookup from multi-step composition, and separate type-correct candidates from candidates whose behavioral contract is established. Report how many outputs require reification, how many abstract negatives can be realized, and how many failures are only unsupported or exhausted.

The same source-to-model adapter should be available to the baseline Lean arm. Otherwise a success attributed to a new type layer may be entirely due to a new semantic library. No present execution here measures Leant itself; the model demonstrates the intended boundary without invoking the Haskell engine or its solvers.

14 How far should Lean's type system be extended?

14.1 A real surface extension need not be a new logic

The recommended Rowan extension consists of property-implicit binders, contract-aware application, relation-indexed results, and scoped requirement inference. These features change the author-visible typing discipline and the elaborator's obligations. Their target types remain existing dependent functions, subtypes, structures, and propositions.

This is not merely a cosmetic renaming of tactics. A contract-aware application decides what evidence must be supplied before a use is valid, which values are fixed, which values may be synthesized, and which result relation subsequent operations may consume. Nevertheless, the kernel sees ordinary terms. The richer authoring interface can therefore be developed and removed without requiring every Lean user to adopt a new foundation.

Refinement-type research provides useful precedents rather than a complete solution to this mathematical workflow. Liquid Types combines type inference and predicate abstraction for an automatically manageable refinement fragment [13]. Lovas and Pfenning study a precise interpretation of logical framework refinements using proof irrelevance, including the nontrivial metatheory such an interpretation requires [14]. Rowan borrows the separation between expressive specifications and bounded inference, but does not claim those systems' metatheorems for Lean or for its own frontend.

14.2 A proof-carrying primitive refinement is a possible experiment

A candidate kernel-level former might have rules of the following shape:

$$\frac{A : \mathsf{Type} \quad P : A \to \mathsf{Prop}}{\mathsf{Ref}(A, P) : \mathsf{Type}}, \qquad \frac{a : A \quad h : P(a)}{\mathsf{pack}(a, h) : \mathsf{Ref}(A, P)},$$

with projections

$$\mathsf{value}(r) : A, \qquad \mathsf{evidence}(r) : P(\mathsf{value}(r)),$$

and the expected reduction of projections on `pack`. Explicit subsumption receives a proof $\forall a, P(a) \to Q(a)$ and builds a value of $\mathsf{Ref}(A, Q)$ with the same carrier.

These basic rules translate directly to Lean's existing subtype construction. They supply no new mathematical expressivity. Their justification would have to be operational: a measured reduction in elaboration overhead, transport size, or poor interaction with dependent coercions that cannot be achieved by ordinary library and elaborator improvements.

One must not claim a runtime advantage simply from erasing proof fields; proof irrelevance and proof erasure are already central to the host's proposition treatment [10]. Nor does a native refinement automatically solve structure ambiguity, source fidelity, or admissible abstraction. Those are problems at other interfaces.

14.3 More aggressive conversion rules have a different cost

An attractive but dangerous next step is to make every provable implication $P \Rightarrow Q$ a conversion between refinements, or every propositional equality a definitional equality. This would couple core conversion to mathematical proof search unless the proof is explicitly supplied through a carefully designed rule. The issue is not the slogan "a timeout means false"; a correct implementation could return unknown. The issue is predictability, completeness of the selected conversion procedure, its interaction with dependency, and the enlarged trusted and metatheoretic surface.

Lean already has substantial conversion rules, including proof irrelevance and eta behavior; their actual implementation is subtler than a naive normalization model [10]. Rowan should not promise a new global conversion principle without a precise calculus, an implementation-level account, and a conservation argument. The present proposal adds no CAS, SMT solver, or language-model oracle to conversion.

A narrower research option is explicitly proof-directed coercion with additional reduction laws for coercion composition. Before adopting it, one must specify which laws are definitional, show typing preservation for the extended calculus, address reduction and conversion interactions, translate accepted proofs into the old theory where claimed, and measure the benefit against optimized existing encodings. A paper sketch of two introduction rules is not that validation.

14.4 The decision ladder

Level	Intended benefit	Decision
Property library	Reusable mathematical lemmas and bundled interfaces.	Implement and make available to all evaluation arms.
Use-site elaboration	Automatic premise routing, stable scopes, better diagnostics.	Rowan's first language-level experiment.
Property-implicit syntax	Shorter expression of the same typed uses.	Add only after the use-site comparison.
Explicit native refinements	Potentially cheaper representation or transport.	Consider only after profiling a concrete bottleneck.
Restricted new conversion laws	Potentially simpler dependent coercion paths.	Separate metatheoretic and implementation study.
Open-ended semantic conversion	Maximum implicit reasoning in the core.	Not part of Rowan's proposed architecture.

This answers the type-system question affirmatively at the authoring layer, while leaving a precisely scoped kernel experiment possible. It avoids both extremes: claiming that new syntax is a new foundation, and claiming that a conservative authoring extension cannot meaningfully change how mathematics is expressed.

15 Conservative interpretation and what it does not prove

15.1 Interpretation of the core

Interpret a Rowan object declaration as a Lean binder or definition, a property row as proofs in the same context, a contract as its quantified proposition, and an authorized construction as a dependent pair or an existential proof according to its computational interface. A method application becomes application of a checked Lean theorem. Equality transport uses the appropriate eliminator; other observations use their registered consumer theorems.

For every compiled assertion node v, retain its source position, context, target T_v, and emitted term p_v. A parent proof uses these checked nodes, not merely their displayed strings. A conditional child that remains open cannot be silently treated as an unconditional parent premise.

Theorem 15.1 (Relative validity of completed core derivations)**.** *Assume the source-to-core translation assigns the intended typed statement to each assertion; each registered method application is validated against its Lean theorem; every producer result and certificate bridge is checked at its sealed target; and the context and axiom policies are respected. Then every completed Rowan assertion has a Lean proof of its interpreted proposition, using no axioms beyond those admitted by the selected Lean environment and policy.*

Proof. Induct on the finite assertion-and-method derivation. A hypothesis node is an explicit binder of the interpreted context. A definition node uses its checked term. Property introduction and weakening use the corresponding supplied proofs and implications. A method node applies its checked theorem to the already interpreted arguments and premise proofs. Construction nodes use the checked witness and its proof. Transport nodes use the stated equality eliminator or relation-specific theorem. Producer leaves are checked at the sealed target. Binding and discharge follow the corresponding Lean rules. Every case constructs an ordinary Lean term of the interpreted node's type, and none introduces an unlisted axiom. □

The theorem is deliberately conditional. It does not prove the parser, renderer, inference implementation, or host kernel correct. The more interesting frontend invariant is precisely why the immutable request boundary is separate from this standard induction. A system can produce valid Lean terms while compiling the wrong mathematical statements.

15.2 What happens when search is incomplete?

Search incompleteness affects whether a proof is found, not the meaning of a successful checked proof. If a provider times out, Rowan can preserve the open obligation, allow a human proof, try another bounded provider, or construct a visibly conditional theorem. It cannot strengthen the result status, add a hypothesis without display, or treat lack of a derivation as negation.

A budget change may alter the selected proof or the chance of completion. It must not alter the sealed statement. A repeated run may find a different witness for an existential request; that is permitted by

the original specification. Once a witness is named and used later, changing it is a source edit that invalidates dependent evidence unless an explicit transport argument is supplied.

15.3 The trust ledger

The trusted mathematical boundary is the chosen Lean kernel and the axioms allowed by the project, together with the interpretation of the source as checked statements. External search is untrusted. A verified checker is trusted through its theorem, not because its implementation is short or written in a favored language. Native execution options require their actual axiom and trust accounting; this proposal introduces no native shortcut.

Publication should retain the exact Lean version, dependency revisions, original target, selected witness when relevant, source and evidence hashes, checker version, and axiom inventory. Hashes identify artifacts; kernel replay establishes their mathematical acceptance. On migration, recheck in the new environment and issue a new receipt. Do not relabel an old receipt as evidence that a changed dependency graph has been checked.

16 Worked development III: a higher-order construction contract

A property-aware language must handle more than arithmetic guards. Return to the FLT tensor-family example [6]. Fix a commutative ring A, a finite index type I, and commutative A-algebras H_i that are finite and free as A-modules. The intended operation returns a chosen algebra W, its finite/free evidence, and a family of equivalences

$$e_T : \mathrm{Hom}_{A\text{-alg}}(W, T) \simeq \prod_{i \in I} \mathrm{Hom}_{A\text{-alg}}(H_i, T)$$

for commutative A-algebras T. It also returns the relation

$$e_{T'}(u \circ f)(i) = u \circ e_T(f)(i) \tag{11}$$

for every A-algebra homomorphism $u : T \to T'$, every $f : W \to T$, and every $i \in I$.

```
given a finite family H of commutative A-algebras
with each H[i] finite and free as an A-module

obtain W, e representing the family H
  requiring the tensor universal-property construction
know W is finite and free over A
know e is natural in the target algebra
```

"Representing" and "natural" here name the exact quantified interfaces above. They are not prose-to-proof guesses. In particular, a collection of individually bijective maps e_T cannot be assigned naturality merely from the fact that each component has a type.

A mathematical construction proceeds by finite induction on the family. For the empty family, choose $W = A$; the homomorphism out of A is its specified A-algebra structure map. On adjoining a factor H, replace the previous representative W_0 by $W_0 \otimes_A H$. Its universal property identifies maps into a commutative target T with a pair of maps from W_0 and H. Composing the previous equivalence yields the desired family interface. Naturality follows because postcomposition commutes with restriction to the tensor inclusions, and equality of the resulting maps is checked by that universal property.

Finiteness and freeness are preserved by the tensor construction under the hypotheses on the two factors. Reindexing along a finite equivalence transports the whole family and its naturality, not just a list of vertex properties.

This is the structure of the source argument, not a new formalization run. The intended Rowan library should provide the finite-induction and tensor adapters as ordinary Lean results; the elaborator merely assembles their contracts. It should preserve the original universe and structure arguments and record that W is a newly chosen witness.

The negative neighbor removes finite/free assumptions on the factors but keeps the finite index. For a singleton with $A = \mathbb{Q}$ and $H = \mathbb{Q}[X]$, a representing algebra with the stated naturality is isomorphic to H, whereas the monomials $1, X, X^2, \ldots$ are linearly independent over $\mathbb{Q}$. A finite-dimensional representing algebra is therefore impossible. The missing premises are mathematically substantive. No amount of notation compression repairs their omission.

This example also separates noncomputable mathematical existence from an executable construction. A proof that the index is finite need not present an enumeration to a running CAS. An executable version asks for explicit finite data; an existence proof may use the host's allowed noncomputable principles and report them in its policy. Rowan should make that interface choice visible.

17 What the accompanying model actually executed

17.1 Scope of the executable artifact

The file `rowan_model.py` contains two connected experiments. The first is a small sorted symbolic requirement engine with a literal parser for `object`, `assume`, and `derive`. It validates schema shapes, grounds them over the declared term pool, builds a backward cone, computes forward closure, and replays each successful derivation against the original request. It tracks scope, snapshot identity, permitted support, and permitted rule names. These are symbolic model checks; registered rules are assumed axioms of that model.

The second experiment is an executable interpreter and affine normalizer for natural-list expressions, together with the semilinear equality procedure of Theorem 11.4. Exact integer arithmetic computes checks and witness coordinates. The code validates arities before combining data. It does not infer a source image, prove its coverage, or accept a model witness as a Lean refutation.

The richer Rowan listings elsewhere in the article are design examples. They are not input accepted by this parser. The package includes no Lean plugin, Haskell bridge, installed prover, or claimed newly compiled Lean file.

17.2 The parsed demonstration

The actual demonstration begins:

```
object a : Int
object b : Int
object p : Nat
assume even_b : Even(b)
assume bound_p : AtLeastFour(p)
derive fourth_coefficient : ExactA4(a,b,p)
derive second_coefficient : ExactA2(a,b,p)
```

```
assume odd_p : Odd(p)
assume residue_a : ResidueThree(a)
derive second_coefficient_completed : ExactA2(a,b,p)
```

The first result is derived from two initial assumptions. The second remains open with `Odd(p)` and `ResidueThree(a)` as its frontier. The third is derived after those hypotheses are explicitly supplied. The model does not invent them. The full result records ten ground assignments for each request; only two ground rules are relevant for the fourth coefficient and one for the second. The example's exact integer evaluator separately returns -53 and -486 at the positive fixture.

The demonstration also compares n_1 and n_2 on the diagonal image and on the full two-dimensional image. It accepts the former and returns $(1, 0)$ for the latter. These two outputs make the distinction between correlated and independent observations inspectable without any solver heuristics.

17.3 Executed tests and generated cases

The companion `test_rowan.py` was executed under Python 3.13.5. All 32 named tests passed. The retained log and machine-readable result file are the authority for these counts.

Executed check	Count	Scope
Named tests	32	No failures or errors.
List/interpreter length comparisons	12,675	507 depth-at-most-two expressions, each on 25 length pairs.
Affine pair/image decisions	4,374	27 forms against 27 forms on six supplied images.
Finite affine sample evaluations	24,057	Cross-checks including the bases and unit-generator coordinates.
Shuffled-rule trials	100	Target acceptance and derivation replay under reordered rules.
Frey arithmetic fixtures and neighbors	139	135 positive arithmetic triples and four single-premise negative neighbors.
Explicit realized list counterexamples	2	Constant-empty and doubled-input length failures on natural lists.

The named symbolic tests include unseeded and seeded cycles, sibling-scope leakage, parent-scope reuse, rebuilt snapshot rejection, identical displays with different meaning identities, explicit rather than string-based transport, forbidden support, forbidden rules, a returned proof of the wrong target, forged leaves and rules, transitive support replay, grounding caps, malformed predicate applications, repeated premises, and nullary rules.

The affine tests include the empty element type, independent and diagonal images, a shifted correlated image, finite unions, the empty admissible domain, malformed dimensions, and returned witness membership. The finite sample cross-check is not the generic completeness proof. That proof is the argument in Theorem 11.4; it has not been mechanized here.

These counts are not independent Lean theorems, proof obligations from a real file, or human-study observations. The one recorded runtime is a single toy execution and is not used as a speed comparison with Lean, Leant, or a CAS.

17.4 A limitation revealed by the model's simplicity

The model makes authority boundaries testable, but its term sorts and meaning identities are explicit strings, not Lean expressions checked in a dependent type theory. Its request immutability is enforced by ordinary Python data structures and API usage, not a security boundary against arbitrary malicious Python code. A caller can also supply a mathematically false registry axiom; the engine would then derive its consequences faithfully.

That limitation is intentional and precisely identifies the next integration work: replace symbolic theorem assumptions with checked Lean registrations, replace the sorted term pool with well-scoped Lean expressions, and replace the derivation replay's symbolic final check with Lean's original-target check. Neither a larger parser nor more random test cases supplies those missing semantic connections.

18 A Lean implementation and evaluation that could reject Rowan

18.1 Build the narrowest meaningful vertical slice

The first Lean package should have a conventional mathematical library, an obligation service, and a thin optional authoring layer. The concrete first use site is the exact-quotient transport, because it has a satisfiable fixture, independent premise groups, short mathematical proofs, and nearby false variants. Reuse the prior checked arithmetic artifacts where available rather than rewriting them from a remembered theorem name.

Implement a property-implicit argument mechanism for the divisibility proof, a validated registry of a few implications, the fixed-request boundary, and an inspectable frontier. Compare it with an ordinary Lean helper invoking the same arithmetic and search services. Then add the local equality-to-derivative adapter. This tests a relation-indexed demand rather than only unary facts.

For the behavioral slice, mechanize the list interpreter and its denotation, the semilinear base-and-generator criterion, and the positive/negative source transfer interfaces. Add one exact candidate reifier for the small grammar. Require a returned negative witness to pass the original source interpretation and its admissibility predicate. Only then connect a Leant candidate handoff. The wider SMT and provider ecosystem can remain explicitly outside this first verified fragment.

A renderer can be built over the resulting typed record. The broader mathematical syntax should wait until these interfaces have demonstrated a benefit. An article-sized vocabulary is not an acceptance criterion.

18.2 Matched arms separate three possible explanations

Use the reports' controlled comparison, with the resources shared fairly [1, 2]:

Arm	Available facilities
L_0	Ordinary Lean with its existing automation.
L_1	L_0 plus the new mathematical packages and relevant registrations.
L_2	L_1 plus the same search providers, certificate services, and semantic adapters used by Rowan.
L_3	L_2 plus Rowan's use-site property interface, finite inference, and frontier.
L_4	L_3 plus the compact mathematical authoring syntax.
L_{2R}	L_2 plus a read-only compact renderer, without Rowan input syntax.

The comparison $L_3 - L_2$ isolates the proposed type-and-obligation layer. $L_4 - L_3$ isolates input syntax. L_{2R} tests whether most of the reading benefit can be obtained without a new language. Infrastructure development and maintenance costs must be included, with reuse and amortization reported rather than assumed.

The inspected source examples are development material, not held-out tests. A credible study should freeze the prototype and its registry before selecting additional tasks, include already-short Lean controls, and stratify by domain and author experience. A counterbalanced within-author design can reduce learning and individual-skill effects, but sample size and power must be chosen from actual pilot variance, not invented in a proposal.

18.3 Measure mathematical work and mathematical misunderstanding

Record author interventions, time to a checked proof, failed-edit diagnosis, repair after library changes, and the ability of another reader to reconstruct the exact statement and premises. Measure elaboration time, proof size, index size, grounding attempts, rules actually used, and time spent in host checking separately from provider search. A shorter display alone does not show a productivity improvement.

Use paired mutations: remove openness; specialize the induction hypothesis too early; omit divisibility; change the coefficient ring; change the measure; drop a factor's finite/free premise; substitute an impossible abstract input; change a candidate's identity; and retain stale evidence after rebuilding a context. Each test needs a positive fixture, the admitted assumptions, the mutation, and an expected *kind* of failure. Missing proof, false claim, invalid method, and stale artifact are not interchangeable labels.

Readers should be asked whether a displayed equality is global or local, whether an integral identity is conditional, whether a candidate law is universal or sampled, and whether a bound is proved least. These questions test the danger created by compression rather than merely reader preference for attractive notation.

18.4 Stop conditions and honest outcomes

If L_1 explains the gain, ship a property library. If L_2 explains it, ship the search and certificate services. If L_{2R} explains the reading benefit without increasing misunderstanding, ship a renderer. Adopt a new use-site language only when its incremental benefits outweigh its costs and its error rates are acceptable.

The kernel experiment has a separate gate: demonstrate a concrete cost in the best ordinary encoding, implement the smallest explicitly specified extension, establish the claimed conservation properties, and compare again. A proposal should be allowed to conclude that the existing kernel was already the right one.

19 Answers to the next-round questions and remaining risks

The reconciled second report lists ten questions for a fourth iteration [2]. Rowan answers some by design or written proof, some by a small execution, and leaves others as measurements rather than pretending that another design document has supplied them.

Item	Question	Rowan's answer and evidence status
T1	Evidence-index cost on a real file	Not measured. The toy records ten ground instances in the demo; it does not estimate Lean file costs.
T2	Which registered rules fire?	Exact model trace: two for the fourth coefficient, one for the second. Real-library overlap with ordinary lookup remains to measure.
T3	Does rewriting break anchors?	Reuse only by host conversion or explicit typed transport; rebuild after edits. Symbolic mismatch/transport tests execute, Lean integration does not.
T4	Shared reification	Share the original-term/interpretation/bridge interface; share implementations only across genuinely common grammars.
T5	Completeness of abstractions	A written complete criterion for affine equality on supplied semilinear images, with exact-image and source-bridge hypotheses. No blanket claim for all Length contracts.
T6	Almost-everywhere consumers	Register a measure-specific relation with appropriate integral consumers, not point evaluation or arbitrary uncountable merging.
T7	Lexical structures and preambles	An explicit proposed policy; no shortening measurement is claimed.
T8	Bounded rule generation	A finite typed pool and the bound $\sum_r \prod_i m_{r,i}$; the sorted first-order version executes.
T9	Reader recovery	A specified semantic reading test; no participants or results are invented.
T10	Independent implementations	The fixed request, grounding, and observation interfaces can be implemented independently against shared mutations. Only one reference model is supplied here.

The principal risks are predictable. Grounding may be too large even though finite. Aggressive property inference may duplicate existing automation. Evidence transport may enlarge terms and slow checking. Contract annotations may merely move verbosity from proofs into declarations. The rich display may hide premises that readers fail to inspect. A provider's semantic bridge may require more proof engineering than the saved proof scripts justify.

Rowan's mitigations are deliberately concrete: small profiles, stage-local term pools, disposable indices, explicit relation kinds, shared-library controls, and strict claim-status reporting. They are not guarantees of productivity. A useful negative result would identify exactly which part should survive as an ordinary Lean library or editor feature.

20 Conclusion

Rowan's mathematical style is to name the objects, state their properties, apply an operation through its contract, and expose the unfinished argument. Its type layer preserves the objects, structures, quantifiers, and exact claim.

The fourth-round contribution makes three weak points concrete: seal the original target before search, bound theorem instantiation as well as closure, and check behavioral models on observations that can actually occur. The semilinear affine criterion supplies a complete constructive fragment, including empty-type and correlated-input cases.

Begin with Lean's elaborator and ordinary proofs. Consider a native refinement primitive only after measuring a remaining obstruction. Success is less routine glue without sacrificing an accurate, inspectable mathematical claim.

A Artifact guide and reproduction

File	Purpose
`Rowan.tex`	Self-contained LaTeX article; bibliography included.
`Rowan.pdf`	Compiled article.
`rowan_model.py`	Parsed finite inference model and affine/semilinear checker.
`test_rowan.py`	32 named tests and generated finite cases.
`demo_results.json`	Retained demonstration output.
`test_results.json`	Exact counts, interpreter version, and single-run timing.
`test_log.txt`	Named-test execution log.
`sources.json`	Inspected repository revisions, paths, and source identities.
`README.md`	Build instructions, scope, and trust limitations.
`SHA256SUMS.txt`	Checksums of the packaged artifacts other than the checksum file itself.

Run the model and tests with a recent Python 3 interpreter; no third-party Python dependency is required:

```
python rowan_model.py
python test_rowan.py
latexmk -pdf -interaction=nonstopmode -halt-on-error Rowan.tex
```

The Python sources target Python 3.11 or later and were executed here with 3.13.5. Running the tests rewrites `test_results.json`; its timing will vary. The LaTeX build uses standard packages and requires no external image, font file, bibliography processor, or network access. The archive does not bundle the upstream repositories.

B Source-selection and version register

The reviewed repository sources are identified by immutable revisions below. The long source URLs in the bibliography refer to those exact revisions, not floating branch contents. Full source blob identifiers are in `sources.json`.

Repository	Inspected revision and scope
Brainstorm	`bf3339059950608 70f8585e297ffc16f3fe7e86`. Close reading of both round-three main reports, including the reconciliation; no claim of an independent full audit of every appendix or individual package.
ProveIt	`24ce8bd743eaab64a91ce90725ea00f498d319d2`. Autonomous derivative-family source, with related files located for context.
Anthropic FLT	`aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`. Frey definitions, integral-model compatibility solution, and tensor-family solution.
Leant	`3a40904be8a410d832d9ab6900b3d3e7b425eccb`. Verification receipt boundary, Length adapter, and post-verification ordering boundary.

The round-three reports' discussed later Leant identity `823259f7` was not retrievable during this review. It is recorded as an availability limitation, not evidence that the reports' account is false. The current online Lean and Mathlib documentation was consulted for design context; it is not a claim that all currently documented APIs exist unchanged at the reports' Lean 4.32.0 pin. No upstream project was built in this session.

References

[1] Brainstorm campaign, *From Properties to Proof Obligations*, round-three synthesis, 6 September 2026. Reviewed at commit `bf333390`. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex.

[2] Brainstorm campaign, *Nine Refinements: Property-Aware Typing in a Mathematician's Proof Language over Lean*, round-three unified review, 6 September 2026, including the reconciliation with the parallel synthesis. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf3333905995060870f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex.

[3] Vladimir Reshetnikov and contributors, ProveIt, `AutonomousIteratedDeriv.lean`, especially `AutonomousODE.iteratedDeriv_eq_comp`. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean.

[4] Anthropic FLT repository, `Def_FLTPrelim_FreyPackage.lean`. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_FLTPrelim_FreyPackage.lean.

[5] Anthropic FLT repository, `S_FreyPackage_freyCurveInt_map.lean`, particularly the coefficientwise exact-division transport. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean.

[6] Anthropic FLT repository, `S_Algebra_exists_algHom_equiv_pi.lean`, finite/free representing algebra with a natural family of equivalences. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean.

[7] Vladimir Reshetnikov and contributors, Leant, `src/Leant/Synth/Verification.hs`, callback-acceptance receipts and quota-aware candidate verification. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs.

[8] Vladimir Reshetnikov and contributors, Leant, `src/Leant/Synth/Length/Adapter.hs`, checked candidate/query provenance and the limits of model-level authority. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Adapter.hs.

[9] Vladimir Reshetnikov and contributors, Leant, `src/Leant/Synth/PostVerification.hs`, epoch-scoped candidate handles and bounded permutation validation. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/PostVerification.hs.

[10] Lean Language Reference, *The Type System* and *Propositions*. Online documentation consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/The-Type-System/; https://lean-lang.org/doc/reference/latest/The-Type-System/Propositions/.

[11] Mathlib documentation, *Mathlib.Tactic.FunProp*. Compositional property proofs, side-condition handling, and theorem registration. Consulted 6 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.

[12] Lean Language Reference, *The mvcgen tactic: Overview*. Predicate transformers, specification lemmas, and verification conditions. Consulted 6 September 2026. https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/.

[13] Patrick Rondon, Ming Kawaguchi, and Ranjit Jhala, *Liquid Types*, PLDI 2008. Author-maintained paper page. https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.

[14] William Lovas and Frank Pfenning, *Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance*, Logical Methods in Computer Science 6(4:5), 2010. https://arxiv.org/abs/1009.1861v4.

[15] Danel Ahman, Cătălin Hriţcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy, *Dijkstra Monads for Free*, POPL 2017. Project paper page. https://fstar-lang.org/papers/dm4free/.

[16] John Harrison and Laurent Théry, *A Skeptic's Approach to Combining HOL and Maple*, Journal of Automated Reasoning 21, 279–294, 1998. Author-maintained paper page. https://www.cl.cam.ac.uk/~jrh13/papers/cas.html.